LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
INHERITANCE (PEWARISAN CLASS)



Identitas Mahasiswa :

Nama	: GNAYAS AL HARITS
NIM	: 2530521067
Kelas	: 3 TI C

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS MUHAMMADIYAH SUKABUMI
2026

1. Tujuan dan Ringkasan Konsep

Tujuan Praktikum

Praktikum Pertemuan 6 bertujuan agar mahasiswa mampu menjelaskan hubungan is-a dan membedakannya dari has-a (komposisi), mendefinisikan parent class dan child class dengan extends, mengidentifikasi method yang diwariskan beserta akses public, protected, dan private, memakai parent::__construct() untuk menginisialisasi bagian parent, serta menerapkan method overriding dan parent::namaMethod(). Seluruh konsep diterapkan pada pengembangan CRUD Data Mahasiswa dari Pertemuan 5 agar dapat menyimpan dua jenis mahasiswa, yaitu Reguler dan Beasiswa, tanpa menggandakan logika dasar.

Ringkasan Konsep Inheritance

Inheritance berarti class anak (child/subclass) memperoleh property dan method yang dapat diwariskan dari class induk (parent/superclass). Aturan umum diletakkan di parent, sedangkan perbedaan diletakkan di child. Pada praktikum ini MahasiswaReguler adalah Mahasiswa dan MahasiswaBeasiswa juga adalah Mahasiswa; keduanya memiliki NIM, nama, fakultas, prodi, asal sekolah, semester, hobi, dan agama, tetapi memiliki aturan biaya yang berbeda.

Hubungan antarclass hanya layak memakai inheritance bila benar-benar is-a. Bila sebuah class hanya memiliki atau memakai objek lain, misalnya Mahasiswa memiliki Alamat, hubungan tersebut dimodelkan dengan komposisi (has-a). PHP hanya mendukung satu parent pada extends; trait adalah mekanisme reuse kode, bukan multiple inheritance.

Keterkaitan dengan Pertemuan Sebelumnya

Pertemuan	Keterkaitan dengan Pertemuan 6
Pertemuan 3 (class, object, CRUD)	Dasar pembuatan class Mahasiswa dan operasi tambah, tampil, edit, hapus yang kembali dipakai pada CRUD.
Pertemuan 4 (constructor, destructor, overloading)	Constructor child memanggil constructor parent lewat parent::__construct(). Overriding pada Pertemuan 6 dibedakan dari overloading (__get/__set/__call) pada Pertemuan 4.
Pertemuan 5 (enkapsulasi)	Seluruh property tetap private dengan getter, setter berisi validasi, dan NIM tanpa setNim(). Repository berbasis PHP Session, daftar agama yang diizinkan, serta pesan error Pertemuan 5 dipertahankan dan hanya diperluas dengan jenis mahasiswa.

2. Cuplikan Kode Sebelum dan Sesudah Refactoring

Sebelum Refactoring (Pertemuan 5)

Hanya ada satu class Mahasiswa untuk semua mahasiswa. Seluruh property sudah private dan divalidasi lewat setter, tetapi tidak ada konsep jenis mahasiswa maupun biaya kuliah.

```
class Mahasiswa {                    // satu class untuk semua mahasiswa
    private string $nim;   private string $nama;
    private string $fakultas; private string $prodi;
    private string $asalSekolah; private int $semester;
```

```
    private string $hobi;  private string $agama;
    // constructor, getter, setter + validasi (Pertemuan 5)
}
$m = new Mahasiswa($nim, $nama, $fakultas, $prodi, $asalSekolah, $semester, $hobi, $agama);
$repo->tambah($m);                    // tidak ada jenis mahasiswa / biaya
```

Sesudah Refactoring (Pertemuan 6)

Class Mahasiswa dijadikan abstract sebagai parent. Data identitas dan validasi Pertemuan 5 tetap berada di parent sehingga tidak diulang. Perbedaan UKT dan potongan beasiswa dipindahkan ke dua child, dan method hitungBiaya() serta getJenis() diwajibkan lewat method abstract.

```
abstract class Mahasiswa {                // parent: data umum + validasi (tetap private)
    private string $nim;  /* ... 7 property lain ... */
    public function __construct(string $nim, /* ... */ string $agama) { /* sama seperti P5 */ }
    abstract public function hitungBiaya(): int;
    abstract public function getJenis(): string;
}

final class MahasiswaReguler extends Mahasiswa {
    private int $ukt;
    public function __construct(/* 8 data umum */, int $ukt) {
        parent::__construct(/* 8 data umum */);   // bagian parent diisi oleh parent
        $this->setUkt($ukt);
    }
    public function hitungBiaya(): int { return $this->ukt; }
    public function getJenis(): string { return 'Reguler'; }
}

final class MahasiswaBeasiswa extends Mahasiswa {
    private int $ukt; private int $potonganPersen;
    public function hitungBiaya(): int {
        return intdiv($this->ukt * (100 - $this->potonganPersen), 100);
    }
    public function getJenis(): string { return 'Beasiswa'; }
}

$m = new MahasiswaBeasiswa(/* 8 data umum */, $ukt, $potongan);
$repo->tambah($m);                    // parameter bertipe Mahasiswa: kedua child diterima
<td><?= aman($m->getJenis()) ?></td>    // index.php: satu kode untuk kedua jenis
<td><?= aman(number_format($m->hitungBiaya(), 0, ',', '.')) ?></td>
```

Penjelasan Perubahan

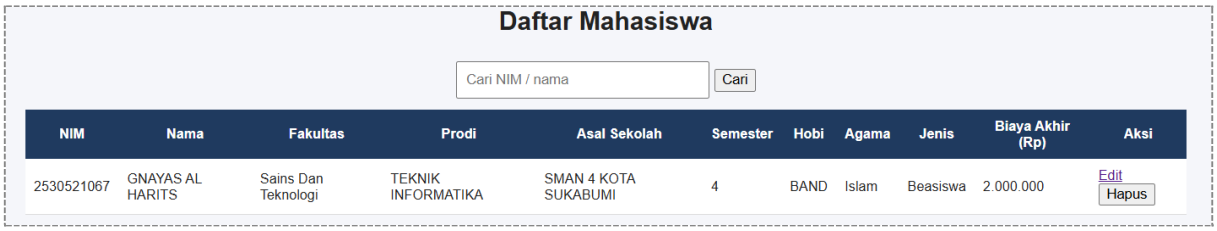
Karena parent bersifat abstract, aplikasi tidak dapat lagi membuat objek Mahasiswa umum; harus dipilih Reguler atau Beasiswa. DataMahasiswa.php hampir tidak berubah karena method tambah() menerima parameter bertipe Mahasiswa yang juga menerima kedua child, dan method ubah() ditambah pemeriksaan agar NIM tidak berubah saat edit. index.php menambah pilihan Jenis, kolom UKT dan Potongan, serta fungsi buatMahasiswa() yang membuat object child sesuai pilihan. Tabel cukup memanggil hitungBiaya() tanpa memeriksa jenis objek. Kolom NIM pada Edit tetap read-only seperti Pertemuan 5, sedangkan jenis boleh diganti.

3. Screenshot Aplikasi CRUD dan Tabel Hasil Pengujian

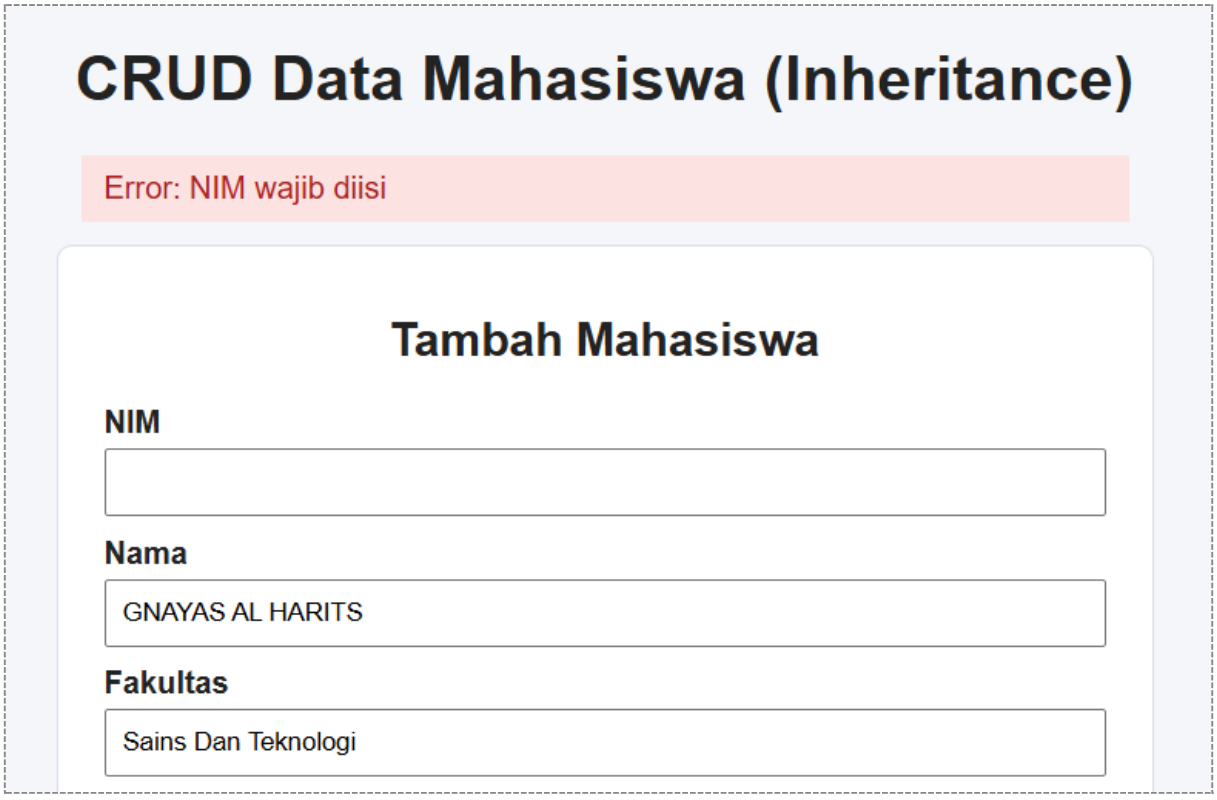
Pengujian dilakukan pada aplikasi CRUD Mahasiswa berbasis PHP Session (tanpa database) yang dijalankan melalui XAMPP di http://localhost/BELAJAR-OOP2/praktikum6/pbo06-inheritance/index.php.



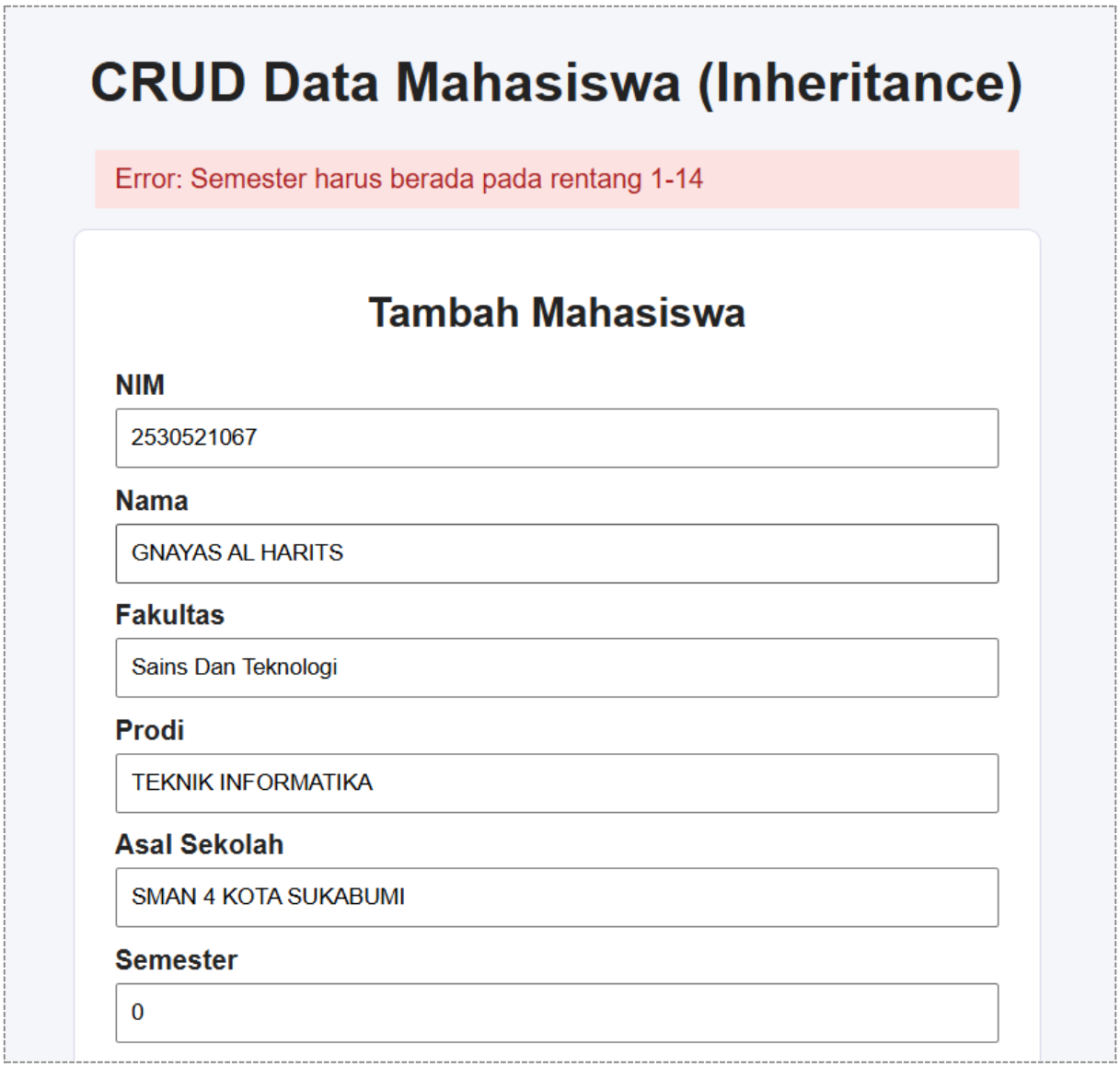
Gambar 1 Mahasiswa Reguler valid, biaya akhir 5.000.000



Gambar 2 Mahasiswa Beasiswa valid, biaya akhir 2.000.000



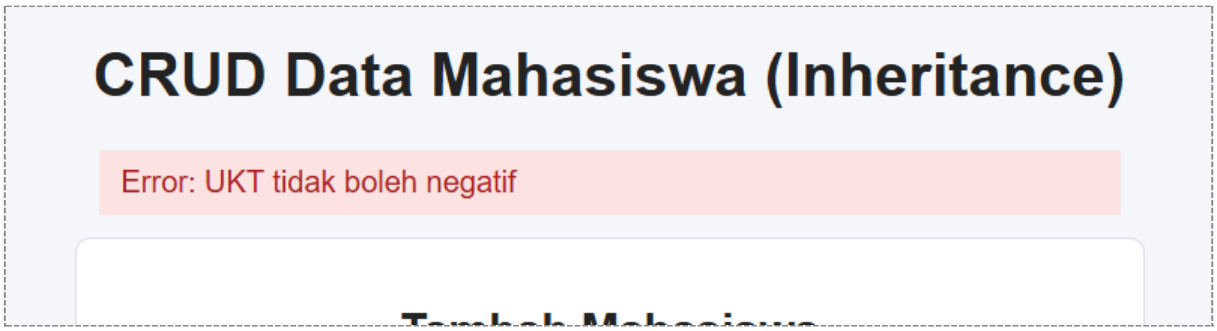
Gambar 3 NIM kosong



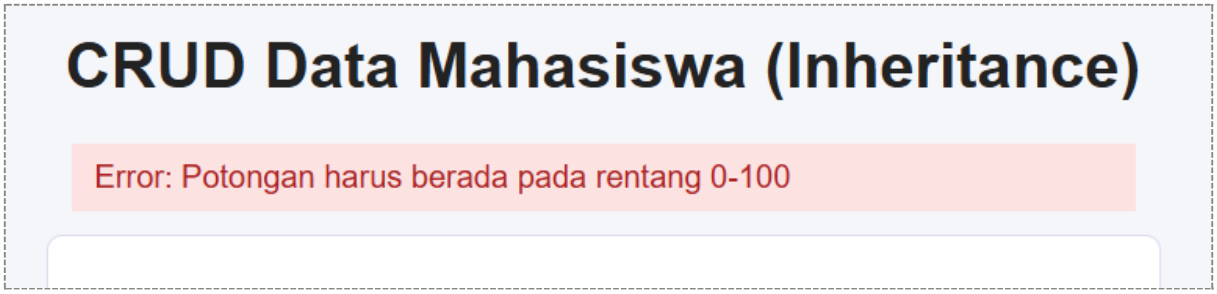
Gambar 4 Semester 0



Gambar 5 Semester 15



Gambar 6 UKT -100 (Reguler)



Gambar 7 Potongan 120%



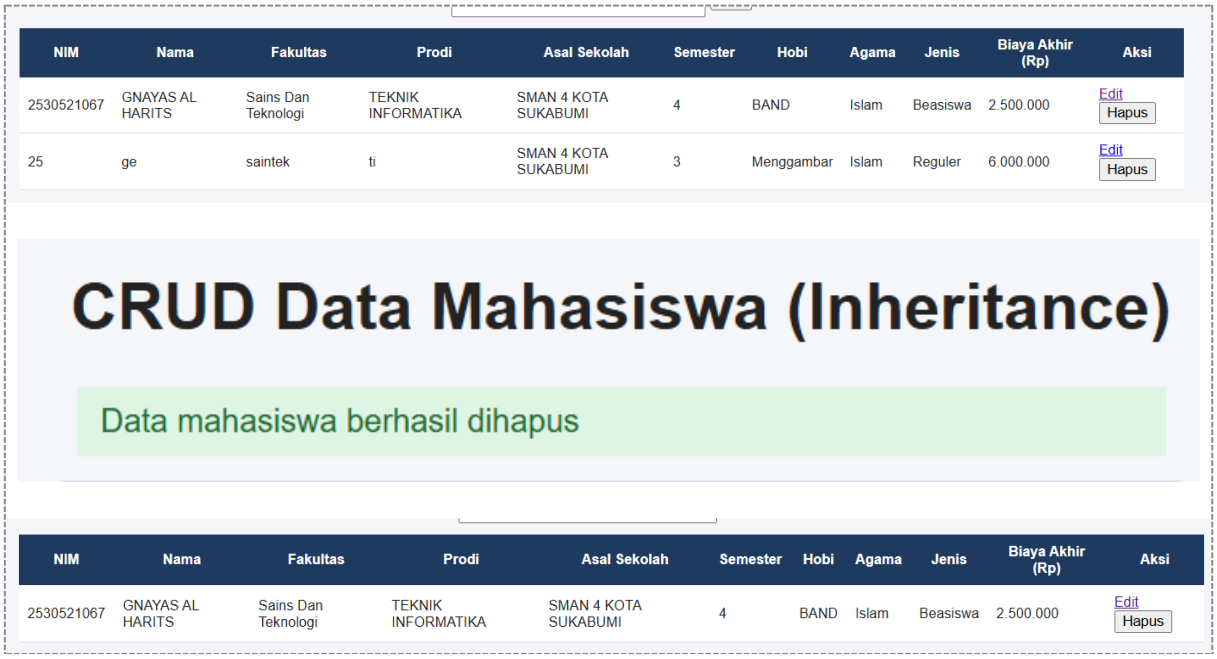
Gambar 8 NIM duplikat



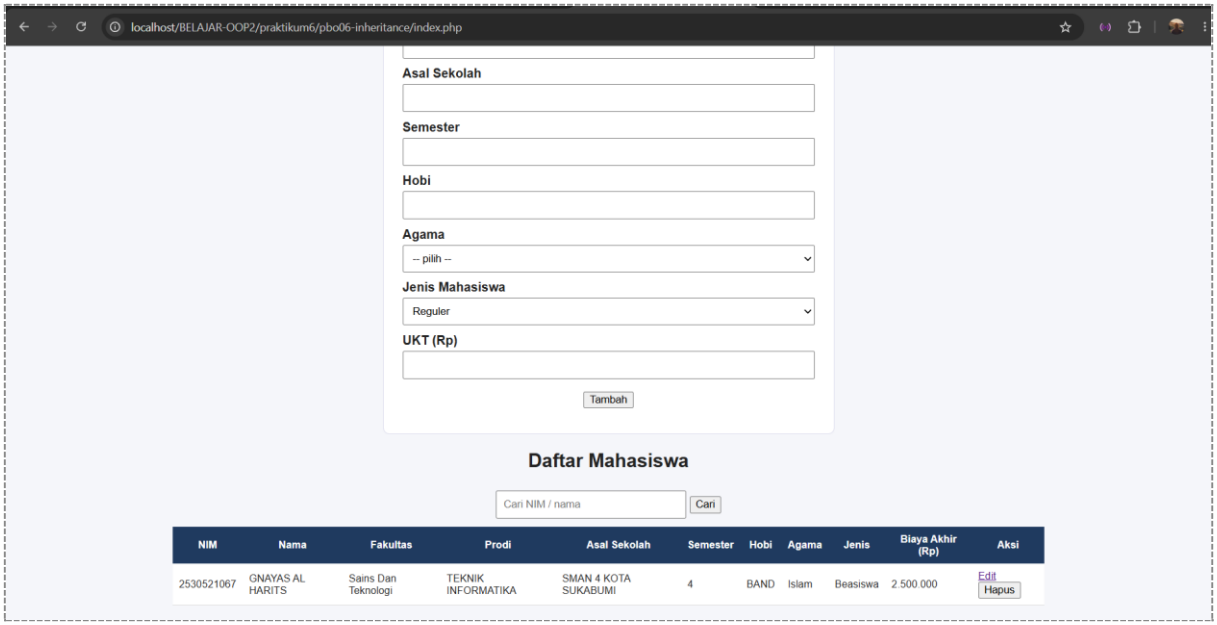
Gambar 9 Edit Reguler menjadi Beasiswa pada NIM yang sama



Gambar 10 "Coba mengubah kolom NIM pada form Edit" dengan hasil "Kolom NIM read-only sehingga tidak dapat diubah dari form"



Gambar 11 Hapus mahasiswa



Gambar 12 Data tetap ada setelah refresh



Gambar 13 Akses langsung property private

Tabel Hasil Pengujian

No	Kasus Uji	Input/Aksi	Hasil yang Diharapkan	Status
01	Tambah Reguler	UKT 5.000.000, semua kolom valid	Tersimpan di Session; biaya akhir 5.000.000; pesan "Data mahasiswa berhasil ditambahkan".	Sesuai
02	Tambah Beasiswa	UKT 5.000.000, potongan 60%	Biaya akhir 2.000.000 (5.000.000 × 40 / 100).	Sesuai
03	NIM kosong	Kolom NIM dikosongkan lalu simpan	Ditolak constructor parent, pesan "Error: NIM wajib diisi".	Sesuai
04	Semester 0 atau 15	Isi semester dengan 0, lalu 15	Ditolak setter parent, pesan "Error: Semester harus berada pada rentang 1-14".	Sesuai
05	UKT -100	Reguler dengan UKT -100	Ditolak setter child, pesan "Error: UKT tidak boleh negatif".	Sesuai
06	Potongan 120%	Beasiswa dengan potongan 120	Ditolak setter child, pesan "Error: Potongan harus berada pada rentang 0-100".	Sesuai
07	NIM duplikat	Tambah dengan NIM yang sudah ada	Ditolak repository, pesan "Error: NIM sudah terdaftar".	Sesuai
08	Edit Reguler menjadi Beasiswa	Ubah jenis pada NIM yang sama	Jenis berubah menjadi Beasiswa; data tetap satu record.	Sesuai

No	Kasus Uji	Input/Aksi	Hasil yang Diharapkan	Status
09	Edit dengan NIM berbeda	NIM diubah lewat inspect element	Ditolak repository, pesan "Error: NIM tidak boleh diubah".	Sesuai
10	Hapus mahasiswa	Klik Hapus pada satu baris	Hanya record terkait yang terhapus; record lain tetap tersedia.	Sesuai
11	Refresh browser	Muat ulang halaman setelah tambah data	Data tetap tersedia selama Session berlaku.	Sesuai
12	Akses langsung property private	Skrip uji: echo $m->ukt; dan echo $m->nim;	$m->ukt: Fatal error Cannot access private property. $m->nim: Warning Undefined property (private milik parent tidak terlihat dari luar).	Sesuai

4. Analisis Kasus

Analisis Pewarisan dan Polimorfisme pada Tabel

Tabel pada index.php memanggil $m->hitungBiaya() dan $m->getJenis() untuk setiap object di Session tanpa memeriksa instanceof. Hal ini berjalan karena Mahasiswa mendeklarasikan kedua method tersebut sebagai abstract, sehingga setiap child wajib mengimplementasikannya. Saat dijalankan, PHP memakai implementasi milik objek sebenarnya: MahasiswaReguler mengembalikan UKT penuh (5.000.000), sedangkan MahasiswaBeasiswa mengembalikan 2.000.000 untuk UKT 5.000.000 dan potongan 60%.

Analisis Akses Private pada Hierarki Class

Property ukt dideklarasikan private pada MahasiswaReguler, sehingga echo $m->ukt; dari luar class menghentikan program dengan Fatal error: Cannot access private property MahasiswaReguler::$ukt. Berbeda dengan itu, property nim dideklarasikan private pada parent. Pada objek child, property tersebut tidak terlihat sama sekali, sehingga echo $m->nim; dari luar tidak menghasilkan Fatal error melainkan Warning: Undefined property dan nilai kosong. Kedua kasus menunjukkan hal yang sama: data tidak dapat dibaca langsung dan harus lewat getNim() atau getUkt(). Child juga tidak dapat memakai $this->nim, sehingga ia memakai getNim() yang disediakan parent.

Analisis Validasi Berlapis

Validasi dibagi menurut pemilik datanya. Parent memvalidasi data umum (NIM wajib, nama dan prodi tidak kosong, semester 1-14, agama harus ada dalam AGAMA_VALID) lewat constructor dan setter yang sama dengan Pertemuan 5. Child memvalidasi data khususnya (UKT tidak negatif dan potongan 0-100). Repository memvalidasi aturan antar object (NIM unik saat tambah, NIM tidak berubah saat ubah). Setiap pelanggaran melempar InvalidArgumentException yang ditangkap di index.php dan ditampilkan sebagai Error di atas form. Constructor child memanggil parent::__construct() terlebih dahulu, sehingga data tidak valid pada bagian parent ditolak sebelum child menyimpan apa pun.

Analisis Edit dengan Pergantian Jenis

PHP tidak dapat mengubah class sebuah object yang sudah ada. Karena itu saat Edit, object baru dibuat lewat buatMahasiswa() dengan NIM yang sama (diambil dari kolom read-only), lalu repository mengganti object lama pada key NIM tersebut lewat ubah($nimLama, $m). Object baru hanya disimpan bila seluruh validasi lolos, sehingga data lama tidak berubah setengah-setengah ketika salah satu isian ditolak, sama seperti tujuan penggunaan clone pada Pertemuan 5. Bila NIM pada form diubah, repository menolaknya dengan pesan "NIM tidak boleh diubah".

5. Eksperimen Contoh 1–5

Contoh 1 — extends: pewarisan method

```
<?php
class Mahasiswa {
    public function __construct(private string $nama) {}
    public function getNama(): string { return $this->nama; }
}
class MahasiswaReguler extends Mahasiswa {
    public function jenis(): string { return 'Reguler'; }
}
$m = new MahasiswaReguler('Andi');
echo $m->getNama() . ' - ' . $m->jenis();
```

Hasil keluaran:

```
Andi - Reguler
```

Analisis: MahasiswaReguler tidak mendefinisikan getNama(), tetapi tetap dapat memanggilnya karena method public tersebut diwariskan dari Mahasiswa. Constructor parent ikut diwarisi selama child tidak membuat constructor sendiri, sehingga new MahasiswaReguler('Andi') mengisi $nama milik parent.

Contoh 2 — constructor child dan parent::__construct()

```
<?php
class Mahasiswa {
    public function __construct(private string $nim, private string $nama) {}
    public function getNama(): string { return $this->nama; }
}
class MahasiswaBeasiswa extends Mahasiswa {
    public function __construct(string $nim, string $nama,
                                private string $jenisBeasiswa) {
        parent::__construct($nim, $nama);
    }
    public function info(): string {
        return $this->getNama() . ' / ' . $this->jenisBeasiswa;
    }
}
echo (new MahasiswaBeasiswa('23001', 'Siti', 'Prestasi'))->info();
```

Hasil keluaran:

```
Siti / Prestasi
```

Analisis: Karena MahasiswaBeasiswa memiliki constructor sendiri, constructor parent tidak dijalankan otomatis. Pemanggilan parent::__construct($nim, $nama) wajib ditulis agar data umum terisi, sedangkan jenisBeasiswa dikelola oleh child.

Contoh 3 — private versus protected

```
<?php
class Mahasiswa {
    private string $nim = '23001';
    protected string $kategori = 'Mahasiswa';
    public function getNim(): string { return $this->nim; }
}
class MahasiswaReguler extends Mahasiswa {
    public function info(): string {
        // return $this->nim; // SALAH: private milik parent
        return $this->getNim() . ' / ' . $this->kategori;
    }
}
echo (new MahasiswaReguler())->info();
```

Hasil keluaran:

```
23001 / Mahasiswa
```

Analisis: Property $nim yang private tidak dapat dipakai langsung oleh child sehingga child memakai getNim(). Property $kategori yang protected dapat dibaca langsung oleh child tetapi tetap tidak bisa diakses dari luar class. Inheritance tidak menghapus enkapsulasi dari Pertemuan 5.

Contoh 4 — method overriding

```
<?php
class Mahasiswa {
    public function hitungBiaya(): int { return 5000000; }
}
class MahasiswaBeasiswa extends Mahasiswa {
    public function hitungBiaya(): int { return 2000000; }
}
$a = new Mahasiswa();
$b = new MahasiswaBeasiswa();
echo $a->hitungBiaya() . ' | ' . $b->hitungBiaya();
```

Hasil keluaran:

```
5000000 | 2000000
```

Analisis: Method hitungBiaya() pada MahasiswaBeasiswa menggantikan versi parent saat dipanggil lewat object child, sedangkan object parent tetap memakai versi aslinya. Ini overriding, bukan overloading seperti pada Pertemuan 4.

Contoh 5 — parent::method dan trait

```
<?php
trait BisaCetak {
    public function cetak(): string { return 'Siap dicetak'; }
}
class Mahasiswa {
    public function label(): string { return 'Mahasiswa'; }
```

```
}
class MahasiswaReguler extends Mahasiswa {
    use BisaCetak;
    public function label(): string {
        return parent::label() . ' Reguler';
    }
}
$m = new MahasiswaReguler();
echo $m->label() . ' - ' . $m->cetak();
```

Hasil keluaran:

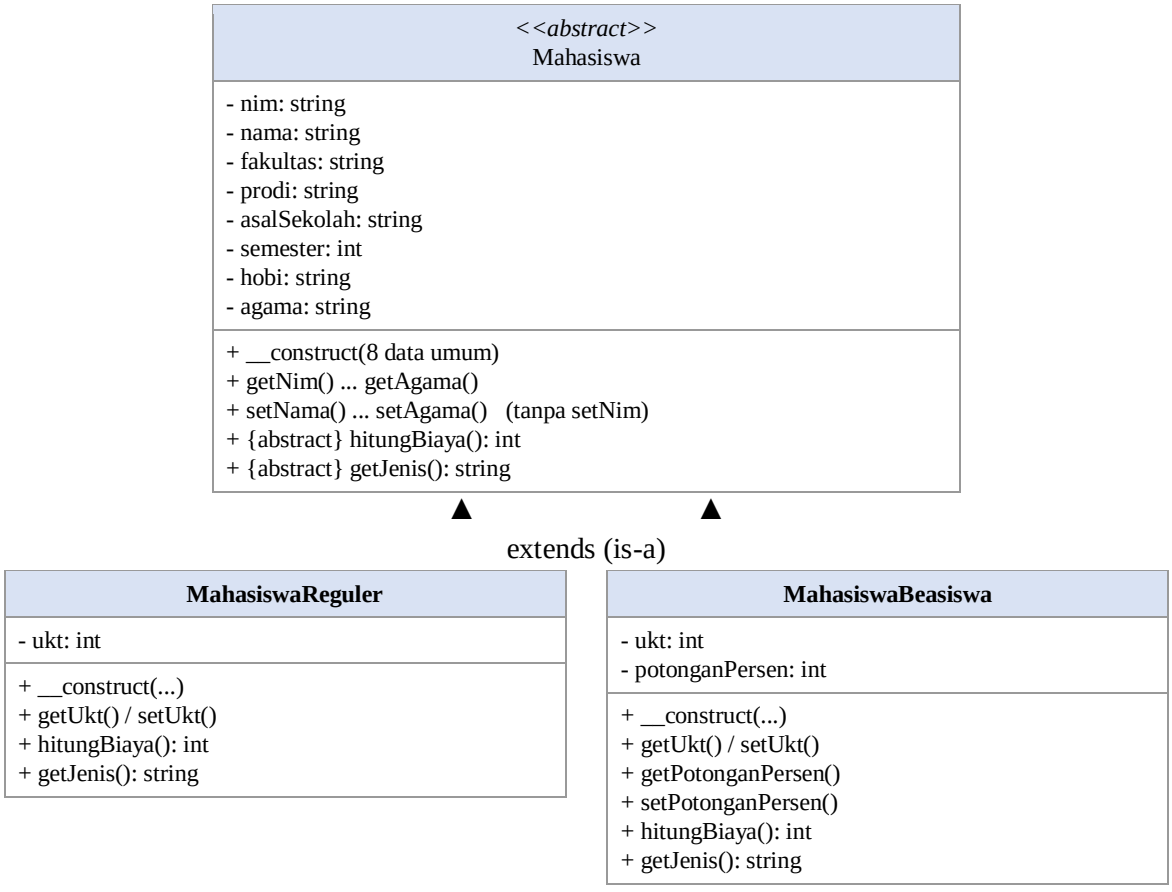
```
Mahasiswa Reguler - Siap dicetak
```

Analisis: parent::label() memanggil implementasi milik parent lalu child menambahkan kata Reguler. Trait BisaCetak hanya menyisipkan method cetak() ke class; ini tidak membuat MahasiswaReguler memiliki dua parent karena extends hanya boleh satu class.

6. Jawaban Tugas Praktikum

Tugas 1 — Diagram class dan penjelasan (individu)

Gambarkan hubungan Mahasiswa, MahasiswaReguler, dan MahasiswaBeasiswa. Tandai property/method yang diletakkan di parent dan child. Jelaskan minimal tiga alasan pemilihan desain. Bandingkan private dan protected pada contoh nyata.



Gambar 14 Diagram class Mahasiswa, MahasiswaReguler, dan MahasiswaBeasiswa

Letak	Property	Method
Parent (Mahasiswa)	nim, nama, fakultas, prodi, asalSekolah, semester, hobi, agama (private)	constructor + validasi, seluruh getter, setter (kecuali NIM), hitungBiaya (abstract), getJenis (abstract)
Child (MahasiswaReguler)	ukt	constructor (memanggil parent), getUkt, setUkt, hitungBiaya, getJenis
Child (MahasiswaBeasiswa)	ukt, potonganPersen	constructor (memanggil parent), getUkt, setUkt, getPotonganPersen, setPotonganPersen, hitungBiaya, getJenis

Alasan pemilihan desain:

1. Hubungan is-a benar: mahasiswa reguler dan beasiswa sama-sama mahasiswa, sehingga extends tepat dipakai.
2. Tidak ada duplikasi: kedelapan data identitas beserta validasinya dari Pertemuan 5 hanya ditulis sekali di parent; child hanya menambah UKT/potongan dan rumus biaya.
3. Parent abstract: objek Mahasiswa umum tidak punya rumus biaya yang lengkap, sehingga aplikasi dipaksa memilih salah satu child.
4. Repository cukup satu: DataMahasiswa menerima parameter bertipe Mahasiswa sehingga kedua child tersimpan tanpa percabangan per jenis.
5. Mudah dikembangkan: jenis baru cukup dibuat sebagai child baru tanpa mengubah parent maupun repository.

Private vs protected pada contoh nyata: Pada Pertemuan 5, setSemester() menolak semester di luar 1-14. Bila $semester dibuat protected, child dapat menulis $this->semester = 99 dan melewati validasi tersebut. Karena itu property identitas tetap private dan child hanya memakai getter. Protected cocok untuk data yang aman dipakai langsung oleh turunan, misalnya $kategori pada Contoh 3, dan sebaiknya dipakai secara selektif.

Tugas 2 — Implementasi inheritance sederhana (individu)

Parent class Pegawai menyimpan id dan nama beserta getNama(). PegawaiTetap menambah gajiPokok dan PegawaiKontrak menambah upahPerJam serta jamKerja. hitungGaji() diimplementasikan berbeda pada tiap child, constructor child memanggil parent::__construct(), dan angka divalidasi tidak negatif.

Kode program :

```
<?php
declare(strict_types=1);
```

```php
abstract class Pegawai {
    public function __construct(private string $id, private string $nama) {
        if (trim($id) === '' || trim($nama) === '') {
            throw new InvalidArgumentException('ID dan nama wajib diisi');
        }
    }
    public function getId(): string { return $this->id; }
    public function getNama(): string { return $this->nama; }

    abstract public function hitungGaji(): int;
    abstract public function getJenis(): string;
}

final class PegawaiTetap extends Pegawai {
    public function __construct(string $id, string $nama, private int $gajiPokok) {
        parent::__construct($id, $nama);
        if ($gajiPokok < 0) throw new InvalidArgumentException('Gaji pokok tidak boleh negatif');
    }
    public function hitungGaji(): int { return $this->gajiPokok; }
    public function getJenis(): string { return 'Tetap'; }
}

final class PegawaiKontrak extends Pegawai {
    public function __construct(string $id, string $nama,
                                private int $upahPerJam, private int $jamKerja) {
        parent::__construct($id, $nama);
        if ($upahPerJam < 0 || $jamKerja < 0) {
            throw new InvalidArgumentException('Upah/jam kerja tidak boleh negatif');
        }
    }
    public function hitungGaji(): int { return $this->upahPerJam * $this->jamKerja; }
    public function getJenis(): string { return 'Kontrak'; }
}

$daftar = [
    new PegawaiTetap('P001', 'Budi', 6000000),
    new PegawaiKontrak('P002', 'Rina', 40000, 100),
];
foreach ($daftar as $p) {
    echo $p->getId() . ' / ' . $p->getNama() . ' / ' . $p->getJenis()
        . ' / Rp' . number_format($p->hitungGaji(), 0, ',', '.') . PHP_EOL;
}
```

Hasil pengujian :



Gambar 15 Hasil pengujian Tugas 2 (dua object pegawai)

Penjelasan : Gaji pegawai tetap sama dengan gaji pokok (6.000.000), sedangkan gaji pegawai kontrak dihitung dari 40.000 × 100 jam = 4.000.000. Perulangan foreach cukup memanggil hitungGaji() tanpa memeriksa jenis objek karena setiap child menaati kontrak method yang sama. Jam kerja dijadikan property agar signature hitungGaji() sama dengan parent.

Tugas 3 — Pengembangan CRUD Mahasiswa (kelompok)

Lanjutkan CRUD Pertemuan 5: form harus dapat memilih Reguler/Beasiswa; data tambahan UKT dan potongan sesuai jenis; tabel menampilkan jenis dan biaya akhir; fitur tambah, cari, edit, dan hapus tetap berfungsi. Edit boleh mengubah jenis tetapi tidak boleh mengubah NIM. Tambahkan validasi UKT ≥ 0 dan potongan 0–100. Tidak diperbolehkan menggandakan kode NIM/nama/prodi/semester pada child.

Kode program :

Mahasiswa.php (ringkas)

```
abstract class Mahasiswa {
   public const AGAMA_VALID = ['Islam','Kristen','Katolik','Hindu','Buddha','Konghucu'];

   private string $nim;   private string $nama;
   private string $fakultas; private string $prodi;
   private string $asalSekolah; private int $semester;
   private string $hobi;  private string $agama;

   public function __construct(string $nim, string $nama, string $fakultas, string $prodi,
                               string $asalSekolah, int $semester, string $hobi, string $agama) {
      $nim = trim($nim);
      if ($nim === '') throw new InvalidArgumentException('NIM wajib diisi');
      $this->nim = $nim;                  // tidak ada setNim()
      $this->setNama($nama);
      $this->setFakultas($fakultas);
      $this->setProdi($prodi);
      $this->setAsalSekolah($asalSekolah);
      $this->setSemester($semester);
      $this->setHobi($hobi);
      $this->setAgama($agama);
   }

   // getter: getNim(), getNama(), getFakultas(), getProdi(),
   //         getAsalSekolah(), getSemester(), getHobi(), getAgama()

   public function setSemester(int $semester): void {
      if ($semester < 1 || $semester > 14)
         throw new InvalidArgumentException('Semester harus berada pada rentang 1-14');
      $this->semester = $semester;
   }
   public function setAgama(string $agama): void {
      if (!in_array($agama, self::AGAMA_VALID, true))
         throw new InvalidArgumentException('Agama tidak termasuk dalam daftar yang diizinkan');
      $this->agama = $agama;
   }
   // setNama(), setFakultas(), setProdi(), setAsalSekolah(), setHobi() mengikuti pola yang sama

   abstract public function hitungBiaya(): int;
   abstract public function getJenis(): string;
}
```

MahasiswaReguler.php

```
require_once __DIR__ . '/Mahasiswa.php';

final class MahasiswaReguler extends Mahasiswa {
   private int $ukt;
```

```
    public function __construct(string $nim, string $nama, string $fakultas, string $prodi,
                                string $asalSekolah, int $semester, string $hobi, string $agama,
                                int $ukt) {
        parent::__construct($nim, $nama, $fakultas, $prodi, $asalSekolah, $semester, $hobi, $agama);
        $this->setUkt($ukt);
    }

    public function getUkt(): int { return $this->ukt; }
    public function setUkt(int $ukt): void {
        if ($ukt < 0) throw new InvalidArgumentException('UKT tidak boleh negatif');
        $this->ukt = $ukt;
    }

    public function hitungBiaya(): int { return $this->ukt; }
    public function getJenis(): string { return 'Reguler'; }
}
```

MahasiswaBeasiswa.php

```
require_once __DIR__ . '/Mahasiswa.php';

final class MahasiswaBeasiswa extends Mahasiswa {
    private int $ukt;
    private int $potonganPersen;

    public function __construct(string $nim, string $nama, string $fakultas, string $prodi,
                                string $asalSekolah, int $semester, string $hobi, string $agama,
                                int $ukt, int $potonganPersen) {
        parent::__construct($nim, $nama, $fakultas, $prodi, $asalSekolah, $semester, $hobi, $agama);
        $this->setUkt($ukt);
        $this->setPotonganPersen($potonganPersen);
    }

    public function getUkt(): int { return $this->ukt; }
    public function getPotonganPersen(): int { return $this->potonganPersen; }

    public function setUkt(int $ukt): void {
        if ($ukt < 0) throw new InvalidArgumentException('UKT tidak boleh negatif');
        $this->ukt = $ukt;
    }
    public function setPotonganPersen(int $potongan): void {
        if ($potongan < 0 || $potongan > 100) {
            throw new InvalidArgumentException('Potongan harus berada pada rentang 0-100');
        }
        $this->potonganPersen = $potongan;
    }

    public function hitungBiaya(): int {
        return intdiv($this->ukt * (100 - $this->potonganPersen), 100);
    }
    public function getJenis(): string { return 'Beasiswa'; }
}
```

DataMahasiswa.php (repository berbasis Session)

```
require_once __DIR__ . '/MahasiswaReguler.php';
require_once __DIR__ . '/MahasiswaBeasiswa.php';

class DataMahasiswa {
    private const KEY = 'data_mahasiswa';

    public function __construct() {
```

```
        if (!isset($_SESSION[self::KEY])) $_SESSION[self::KEY] = [];
    }

    // parameter bertipe Mahasiswa: menerima Reguler maupun Beasiswa
    public function tambah(Mahasiswa $m): void {
        $nim = $m->getNim();
        if (isset($_SESSION[self::KEY][$nim])) {
            throw new InvalidArgumentException('NIM sudah terdaftar');
        }
        $_SESSION[self::KEY][$nim] = $m;
    }

    public function semua(): array { return $_SESSION[self::KEY]; }

    public function cari(string $nim): ?Mahasiswa {
        return $_SESSION[self::KEY][$nim] ?? null;
    }

    // jenis boleh berganti (Reguler <-> Beasiswa), NIM tidak boleh berubah
    public function ubah(string $nimLama, Mahasiswa $m): void {
        if (!isset($_SESSION[self::KEY][$nimLama])) {
            throw new InvalidArgumentException('Data mahasiswa tidak ditemukan');
        }
        if ($m->getNim() !== $nimLama) {
            throw new InvalidArgumentException('NIM tidak boleh diubah');
        }
        $_SESSION[self::KEY][$nimLama] = $m;
    }

    public function hapus(string $nim): void {
        if (!isset($_SESSION[self::KEY][$nim])) {
            throw new InvalidArgumentException('Data mahasiswa tidak ditemukan');
        }
        unset($_SESSION[self::KEY][$nim]);
    }
}
```

index.php (bagian penanganan input dan tampilan)

```
require_once __DIR__ . '/DataMahasiswa.php';   // dimuat sebelum session_start()
session_start();

function buatMahasiswa(array $p): Mahasiswa {
    $semester = filter_var($p['semester'] ?? '', FILTER_VALIDATE_INT);
    $ukt      = filter_var($p['ukt'] ?? '', FILTER_VALIDATE_INT);
    if ($semester === false) throw new InvalidArgumentException('Semester harus berupa angka bulat');
    if ($ukt === false)      throw new InvalidArgumentException('UKT harus berupa angka bulat');
    /* $nim, $nama, $fakultas, $prodi, $asal, $hobi, $agama diambil dari $p */

    if ($p['jenis'] === 'reguler') {
        return new MahasiswaReguler($nim, $nama, $fakultas, $prodi, $asal,
                                    $semester, $hobi, $agama, $ukt);
    }
    if ($p['jenis'] === 'beasiswa') {
        $potongan = filter_var($p['potongan'] ?? '', FILTER_VALIDATE_INT);
        return new MahasiswaBeasiswa($nim, $nama, $fakultas, $prodi, $asal,
                                     $semester, $hobi, $agama, $ukt, $potongan);
    }
    throw new InvalidArgumentException('Jenis mahasiswa tidak dikenal');
}

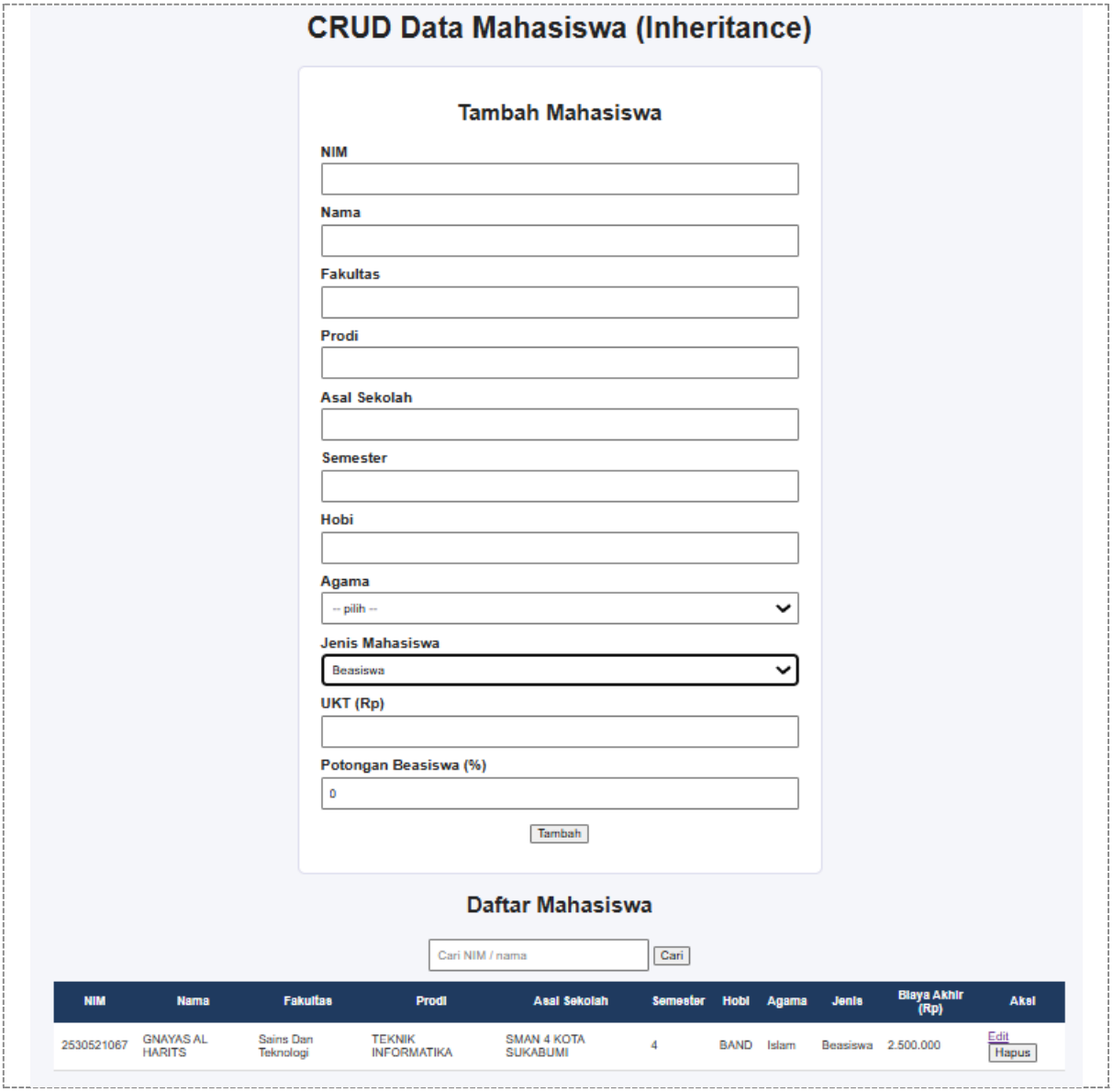
try {
    if ($aksi === 'tambah') {
```

```
        $repo->tambah(buatMahasiswa($_POST));
    } elseif ($aksi === 'ubah') {
        // object baru dibuat dulu; disimpan hanya jika semua validasi lolos
        $repo->ubah($_POST['nim_lama'], buatMahasiswa($_POST));
    } elseif ($aksi === 'hapus') {
        $repo->hapus($_POST['nim']);
    }
} catch (InvalidArgumentException $e) {
    $error = 'Error: ' . $e->getMessage();
}

// pencarian NIM / nama
$tampil = array_filter($repo->semua(), fn(Mahasiswa $m) =>
    $q === '' || stripos($m->getNim(), $q) !== false || stripos($m->getNama(), $q) !== false);

<!-- tampilan tabel: satu kode untuk kedua jenis -->
<td><?= aman($m->getNim()) ?></td>
<td><?= aman($m->getNama()) ?></td>
<td><?= aman($m->getJenis()) ?></td>
<td><?= aman(number_format($m->hitungBiaya(), 0, ',', '.')) ?></td>
<!-- form edit: NIM read-only, jenis boleh diganti -->
<input type="text" name="nim" value="<?= aman($f['nim']) ?>" readonly>
```

Hasil pengujian : Tampilan aplikasi dan seluruh hasil pengujian dapat dilihat pada Bab 3 (Gambar 1 sampai Gambar 13).



Gambar 16 Tampilan aplikasi CRUD Mahasiswa setelah penerapan inheritance

Penjelasan : Pada pengembangan ini kedelapan property Mahasiswa tetap private seperti Pertemuan 5. NIM, nama, fakultas, prodi, asal sekolah, semester, hobi, dan agama berada di parent dan tidak diulang pada child. Child hanya menyimpan data khususnya: MahasiswaReguler menyimpan UKT, sedangkan MahasiswaBeasiswa menyimpan UKT dan potongan persen. Form memilih jenis, tabel menampilkan jenis dan biaya akhir, dan fitur tambah, cari, edit, serta hapus berjalan dengan data tersimpan di PHP Session. Edit boleh mengganti jenis tetapi NIM read-only dan ditolak repository bila diubah. Validasi UKT ≥ 0 dan potongan 0–100 ditegakkan pada setter child.

Tugas 4 — Analisis inheritance vs komposisi (individu)

Hubungan	Jenis	Alasan
Mahasiswa – MahasiswaBeasiswa	is-a	Mahasiswa beasiswa adalah jenis mahasiswa; ia memiliki semua atribut dan perilaku mahasiswa dan dapat dipakai di mana pun Mahasiswa diharapkan.

Hubungan	Jenis	Alasan
Mahasiswa – Alamat	has-a	Mahasiswa bukan sebuah alamat; ia hanya memiliki alamat. Alamat dimodelkan sebagai object terpisah yang disimpan sebagai property.
Mahasiswa – KartuPerpustakaan	has-a	Kartu perpustakaan adalah benda yang dimiliki mahasiswa dan dapat diganti atau dicabut tanpa mengubah identitas mahasiswa.
Mahasiswa – MahasiswaReguler	is-a	Mahasiswa reguler adalah salah satu jenis mahasiswa yang berbagi data identitas yang sama dan berbeda pada aturan UKT.

Contoh desain inheritance yang keliru dan perbaikannya: Menjadikan Mahasiswa turunan Alamat (class Mahasiswa extends Alamat) keliru karena mahasiswa bukan jenis alamat. Mahasiswa mewarisi property dan method alamat yang tidak relevan, dan tidak bisa memiliki lebih dari satu alamat (misalnya alamat KTP dan domisili). Perbaikannya adalah komposisi: Mahasiswa menyimpan object Alamat sebagai property.

```
<?php
// KELIRU: Mahasiswa bukan sebuah Alamat (hubungan has-a dipaksa jadi is-a)
class Alamat {
    protected string $jalan = '';
    protected string $kota = '';
}
class Mahasiswa extends Alamat {
    private string $nama;
}

// PERBAIKAN: komposisi - Mahasiswa MEMILIKI Alamat
class Mahasiswa {
    public function __construct(private string $nama, private Alamat $alamat) {}
    public function getAlamat(): Alamat { return $this->alamat; }
}
```

7. Jawaban Pertanyaan Penguatan Konsep

1. **Mengapa MahasiswaBeasiswa disebut is-a Mahasiswa, bukan has-a Mahasiswa?**

   Karena MahasiswaBeasiswa adalah salah satu jenis mahasiswa: ia memiliki NIM, nama, prodi, semester, dan data lain milik mahasiswa serta dapat dipakai di tempat yang meminta Mahasiswa. Ia tidak "memiliki" objek mahasiswa sebagai bagian dari dirinya, sehingga hubungannya pewarisan, bukan komposisi.

2. **Apa yang terjadi jika child membuat constructor sendiri tetapi lupa memanggil parent::__construct()?**

   Constructor parent tidak dijalankan sehingga validasi NIM, nama, semester, dan agama dilewati serta property private parent tidak pernah diisi. Memanggil getter seperti getNim() kemudian menimbulkan Error karena typed property belum diinisialisasi.

3. **Mengapa child tidak boleh langsung membaca private $nim pada parent?**

Private hanya dapat diakses oleh class yang mendeklarasikannya. Pada child, property tersebut tidak terlihat (hasilnya Undefined property). Aturan ini menjaga enkapsulasi: nilai hanya dibaca lewat getter dan hanya diubah lewat setter yang memvalidasi, sehingga validasi parent tidak dapat dilewati turunan.

4. **Apa beda overriding dengan overloading yang dipelajari pada Pertemuan 4?**

   Overriding terjadi ketika child mendefinisikan ulang method hasil pewarisan dengan nama dan signature yang kompatibel, dan yang dijalankan ditentukan oleh class objeknya. Overloading di PHP berarti penanganan dinamis atas member yang tidak ada atau tidak dapat diakses lewat magic method seperti __get, __set, atau __call; PHP tidak mengenal beberapa method bernama sama dengan parameter berbeda.

5. **Mengapa method hitungBiaya() dapat dipanggil dari variabel bertipe Mahasiswa?**

   Karena Mahasiswa mendeklarasikan hitungBiaya() sebagai method abstract, sehingga setiap objek turunannya pasti memilikinya. Saat dijalankan, PHP memakai implementasi milik objek sebenarnya (Reguler atau Beasiswa).

6. **Apakah use TraitX sama dengan extends ParentX?**

   Tidak. extends membentuk hubungan is-a dengan satu parent dan membuat child bertipe parent, sedangkan use trait hanya menyisipkan method dan property agar kode dapat dipakai ulang, tidak membentuk tipe, dan boleh memakai lebih dari satu trait.

8. Kesimpulan

Inheritance memungkinkan aturan umum mahasiswa diletakkan satu kali di parent, sedangkan perbedaan aturan biaya diletakkan di child. Constructor child bertanggung jawab memanggil parent::__construct() bila ia mendefinisikan constructor sendiri, dan overriding mengubah implementasi method sesuai jenisnya. Penggunaan property private dan validasi setter dari Pertemuan 5 tetap dipertahankan di parent, sedangkan protected sebaiknya dipakai selektif. Pada pengembangan CRUD Mahasiswa, dua jenis mahasiswa tersimpan dalam satu repository tanpa percabangan per jenis, dan input seperti NIM kosong, semester di luar 1-14, UKT negatif, potongan di luar 0-100, serta NIM duplikat berhasil ditolak sebelum masuk ke Session. Hubungan antarclass hanya layak memakai inheritance bila benar-benar is-a; untuk hubungan has-a seperti Mahasiswa dan Alamat lebih tepat memakai komposisi.

9. Link Github

https://github.com/Gnayas/PBO-PERTEMUAN-6